

2024-2025 第 2 学期

《多媒体信息安全技术》期末考查任务总结报告

任务名称：LSB 隐写

姓名与学号

摘要：

本项目对 BMP 格式数字图像进行了最低有效位算法（LSB）隐写并开展分析研究。通过 LSB 隐写算法设计，将隐秘信息隐藏嵌入到 BMP 图像中后进入传输信道，监控者可拦截信息并可通过 LSB 算法检查核验，最后信息传输到接受者处，再通过 LSB 提取算法可得到之前的隐秘信息。实验结果以 Lena 为分析对象，通过卡方分析、信息量估计法和 RS 分析等技术对隐写效果进行检测和评估。实验结果展示了 LSB 隐写在各种分析方法下的检测效果，为图像隐写和隐写分析技术提供了实践参考。

关键词：

空域隐写；BMP 图像；最低有限位；隐写分析

1、任务背景及简介

1.1 隐写术背景及简介

随着互联网的急速发展，人们在通过网络进行数字信息传送和共享的同时，也存在很多安全隐患，如信息的破坏、篡改、丢失等问题也引起各方的重视。

隐写术是一种通过隐藏信息存在性来保障安全的技术，是信息安全领域的一个重要分支，该技术通常以图像、文本、视频等数字产品为载体，根据载体的特征将秘密信息嵌入到载体中，利用载体中存在的冗余信息以达到秘密传递信息的目的，有效地保障了秘密信息安全性。

隐写术嵌入媒体的消息数据通常是加密后的密文，与密码学的加密信息内容形成互补。其核心目标是使第三方无法察觉载体（如图片、音频）中隐藏的信息，两者中间的区别如表 1 所示。

表 1 隐写术与加密的区别

	隐写术隐藏	密码方法加密
数据形式	媒体数据	密文
应用目的	保护数据传递和存储的行为不被发现	保护数据不被对手得到
安全性	抗检测能力	可证明安全性
对抗手段	隐写分析检测	密码分析

1.2 LSB 隐写技术

LSB（Least Significant Bit）即最低有效位，是指图像数据存储中的最低位。其算法具有隐藏容量大，隐蔽性好，对载体改动较小，易于实现的特点，可通过将隐秘信息嵌入到图像像素的最低位上而成功隐藏到图像中，由于人眼对色彩变化不敏感，因此嵌入信息不会对图像视觉效果产生明显干扰，故能在不被第三方检测出的情况下进行传送，从而实现了更隐蔽的数据传输。

该技术隐藏信息的载体优先选择无损压缩格式如 PNG、BMP 等。JPG 是应用最广泛的图像格式之一，由于其采用一种特殊的有损压缩算法，虽然将不易被人们察觉的颜色删除，从而达到较高的压缩比，但这种有损压缩格式会导致画质缺失从而破坏隐藏信息。PNG 与 JPG 格式类似，压缩比高于 GIF,且支持图像透明，画质中等。BMP 相对于 JPG 和 PNG 作为隐藏信息的载体具有结构简单的优势，故能够被大多数软件“接受”，可称为通用格式。此外，BMP 还不采用任何压缩，无损，颜色准确，有 2 色、16 色、256 色、真彩色各种选择，画质最好等多

项优点。

PNG（可移植的网络图形格式）适合于任何类型，任何颜色深度的图片。也可以用 PNG 来保存带调色板的图片。该格式使用无损压缩来减少图片的大小，同时保留图片中的透明区域，所以文件也略大。尽管该格式适用于所有的图片，但有的 Web 浏览器并不支持它。

BMP（Window 标准位图）是最普遍的点阵图格式之一，也是 Window 系统下的标准格式，是将 Window 下显示的点阵图以无损形式保存的文件，其优点是会降低图片的质量，但文件大小比较大。

PNG 格式和 BMP 格式都是一种无损压缩,由于 LSB 隐写就是修改了像素中的最低位缩的图片上实现隐写的，如果是 JPG 图片,就会因为其有损压缩导致无法使用 LSB 隐写。所以，本文将以 BMP 格式数字图像作为隐藏信息的载体。

2、算法原理及实现

2.1 LSB 隐写算法原理

LSB 的数学原理以 RGB 图像为例，每个像素包含三个通道（红、绿、蓝），每个通道的颜色值通常由二进制 8 位（0-255）表示。LSB 隐写利用了人眼对颜色变化的不敏感性，从而将秘密信息嵌入到这些最低位上。其示过程示意如图 1 所示。

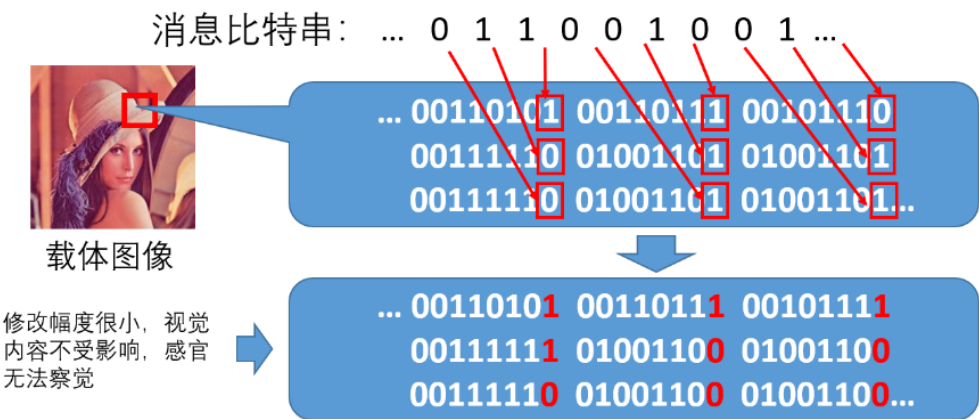


图 1 LSB 算法修改过程示意图

之所以使用最低比特位是因为最低比特位所携带的信息比较少，由于基本上是噪声，故对亮度值的微小变化（如 150→151）几乎无法分辨，便不太对处理后的整体图像呈现产生影响，加之人眼对色彩变化的不敏感，嵌入的信息也不会对

图像的视觉质量产生明显影响，从而实现了隐蔽的数据传输。以案例图的原始图像按低位到高位展开出的 8 个位平面的二值平面图像如图 2 所示。

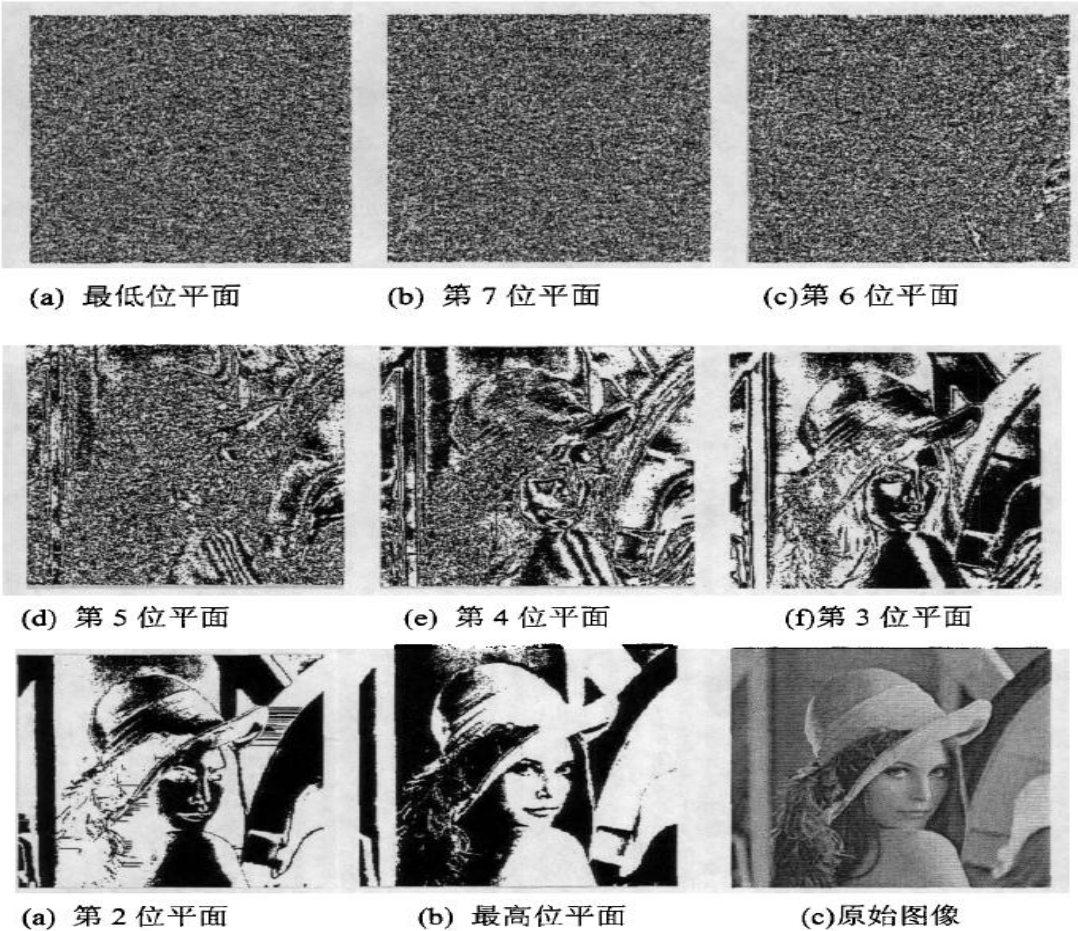
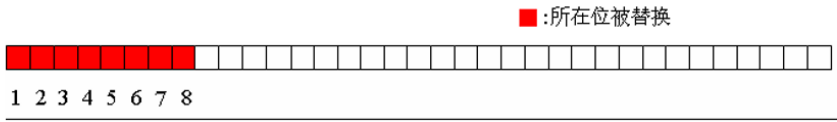


图 2 案例图由低位到高位展开位平面图

LSB 替换隐写可按照嵌入位置选取方法的方法可以分为两种：连续嵌入法和随机间隔法。其替换隐写方法示意如图 3 所示。

连续嵌入法



随机间隔法

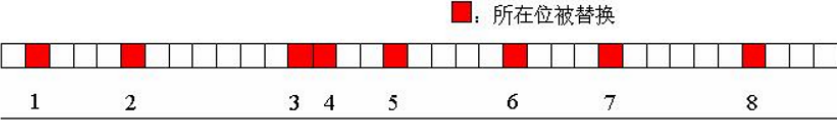


图 3 LSB 替换隐写方法示意图

数据容量计算，若每个像素的 1 个最低位用于隐藏信息，则每像素可存储 1 bit 数据。例如，一张 800×600 的图像最多可隐藏 $800 \times 600 \times 1 = 480,000 \text{ bit} =$

60 KB 数据。

2.2 LSB 隐写算法实现流程

LSB 隐写算法实现流程如图 4 所示。

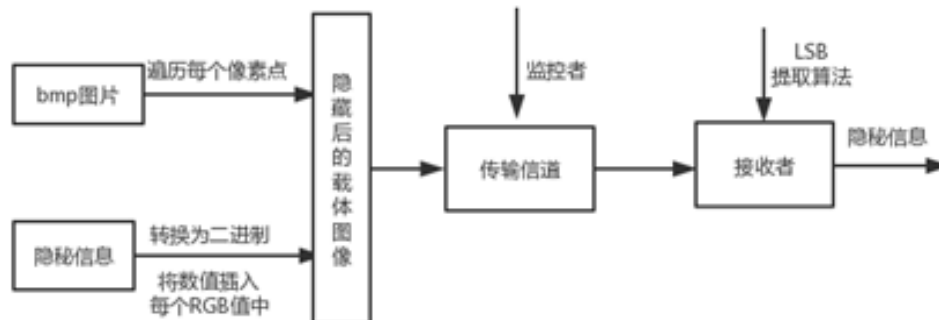


图 4 LSB 隐写算法流程示意图

首先遍历图像中的每个像素点，将图像的像素值存在数组中；

然后将隐秘信息转换为二进制，并且插入每个像素值的最后一位中，通过传输信道传输；

监控者察觉有信息传输接收者，监控者拦截信息查看传输信息中是由可疑信息，然而通过 LSB 算法进写的图像隐写后的载体图像，在内存等方面和原图像比几乎无差别，所以监控者不会怀疑，则信息继续传输；

最后接收者接收到载体图像后，经 LSB 提取算法提取后得到隐秘信息。

2.3 LSB 隐写算法设计

1. 读取秘密文件并转化为比特流

```
with open(secret_file_path, 'r') as file:
    secret_data = file.read()
    binary_secret = ".join(format(ord(char), '08b') for char in secret_data)
```

2. Lsb 顺序隐写

```
for h in range(height):
    for w in range(width):
        pixel = im.getpixel((w, h))
        if count < secret_length:
            pixel = (pixel & ~1) | int(secret_data[count]) #清除最低有效位并嵌入比特
            count += 1
        im.putpixel((w, h), pixel)
        if count >= secret_length:
            break
```

```

if count >= secret_length:
    break

```

3. Lsb 随机隐写

```

pixels_list = [(w, h) for h in range(height) for w in range(width)]
random.shuffle(pixels_list) # 随机打乱像素坐标列表
while count < secret_length and pixels_list:
    w, h = pixels_list.pop() # 获取随机的像素坐标
    pixel = im.getpixel((w, h))
    if count < secret_length:
        pixel = (pixel & ~1) | int(secret_data[count]) # 清除最低有效位并嵌入比特
        count += 1
    # 将修改后的像素写回
    im.putpixel((w, h), pixel)

```

4. 提取秘密信息

```

def extract_lsb_steganography(image_path, secret_length):
    im = Image.open(image_path).convert('L')
    width, height = im.size
    print(width, height)
    secret_data = [] # 用于存储提取的秘密信息
    count = 0 # 用于跟踪已提取的比特数
    for h in range(height):
        for w in range(width):
            pixel = im.getpixel((w, h))
            # 提取最低有效位
            if count < secret_length:
                secret_data.append(pixel & 1)
                count += 1
            # 如果提取完成，则退出循环
            if count >= secret_length:
                break
        if count >= secret_length:
            break
    secret_message = "".join(str(bit) for bit in secret_data)
    return secret_message

```

3、实验结果及分析

3.1 LSB 隐写结果展示

使用 LSB 隐写后携带消息比特串后其图像不可由肉眼看出差异。隐写前后如图 5 所示。



图 5.(a) 案例 1 经 LSB 随机隐写(左图为原图，右图为隐写后的载体图)



图 5.(b) 案例 1 经 LSB 顺序隐写(左图为原图，右图为隐写后的载体图)



图 5.(c) 案例 2 经 LSB 随机隐写(左图为原图，右图为隐写后的载体图)



图 5.(d) 案例 2 经 LSB 顺序隐写(左图为原图，右图为隐写后的载体图)

图 5 案例经 LSB 隐写前后效果图

针对图像可计算其峰值信噪比（PSNR）和香农熵，以此来衡量图像质量指标。一般来说，PSNR 大于 40 dB 通常被认为是较好的图像质量。以连续嵌入隐写信息为例。

对案例 1，隐写图像的峰值信噪比（PSNR）为：51.15 dB，而对案例 2，隐写图像的峰值信噪比（PSNR）为：55.32 dB。结果可知，在隐写场景下的 PSNR 远超常规阈值的 40dB，说明隐写对图像像素级保真度影响极小，由于仅发生在像素层面，故几乎为不可见。由于 PSNR 值高不等同于抗隐写分析能力强，故可再采用香农熵来衡量图像像素值的随机性。

对案例 1，原始图像的香农熵为：7.2338；隐写图像的香农熵为：7.2354，而对案例 2，原始图像的香农熵为：7.0097；隐写图像的香农熵为：7.0161。结果可知，隐写场景下的熵变化增加程度变化微乎其微，大约 0.02% 熵的微小增加表明隐写操作未显著破坏像素分布特征。

计算图像 PSNR 和香农熵代码如下所示：

```
def calculate_psnr(img1, img2):
    mse = np.mean((img1 - img2) ** 2)
    if mse == 0:
        return float('inf')
    return 20 * np.log10(255.0 / np.sqrt(mse))

def calculate_entropy(image):
    hist = cv2.calcHist([image], [0], None, [256], [0, 256]).flatten()
    hist /= hist.sum() # 归一化为概率分布
```



```
hist = hist[hist > 0] # 去除零概率值
entropy = -np.sum(hist * np.log2(hist))
return entropy
```

对隐藏信息采用 LSB 提取算法提取的隐藏信息，使用文本文件的内容作为嵌入数据，则提取出来消息一致。结果如图 6 所示。

```
提取 LSB 顺序隐写...
选择秘密信息来源 (1. 随机生成, 2. 文本文件): 2
256 256
提取的秘密信息 (二进制): 01010100011010000110100101110011001000000110100101110011001000000110
0001001000000111001101100101011000110111001001100101011101000010110000110001001100100011
0011001101000011010100110110001101110011100000111001011000010110001001100011011001000110
01010110011001101010011010000110100101100111011010110110110001101101101110011011110111
0000011100010111001001110011011010001110101011101100111011110000111100101111010
提取的秘密信息 (文本): This is a secret,123456789abcdefghijklmnopqrstuvwxy
提取的秘密信息已保存到文件: extract.txt
```

图 6.(a) 经 LSB 提取算法得到的隐秘信息

```
This is a secret,123456789abcdefghijklmnopqrstuvwxy
```

图 6.(b) 经 LSB 算法写隐前的隐秘信息

图 6 隐秘信息提取结果图

3.2 LSB 隐写结果分析

3.2.1 卡方分析

Westfeld 教授研究发现，在无损存储的图像中通过替换像素值的 LSB 嵌入秘密信息会显著改变图像的直方图，并据此提出了基于像素值对(PoVs)的统计分析方法。像素值对是指在秘密消息嵌入时相互交换的两个值，仅 LSB 不同的两个像素值构成一个像素值对。例如，对于 8bit 灰度图像 0-1, 2(→3, ..., 254、-255 都是值对。Westfeld 发现，在载体图像中，每个像素值对中两个值的出现频次一般不一致，但是在载密图像中，由于嵌入的秘密消息一般具有伪随机序列的统计特性，每个像素值对中两个值的出现频次开始趋向一致，此现象如图 7 所示。

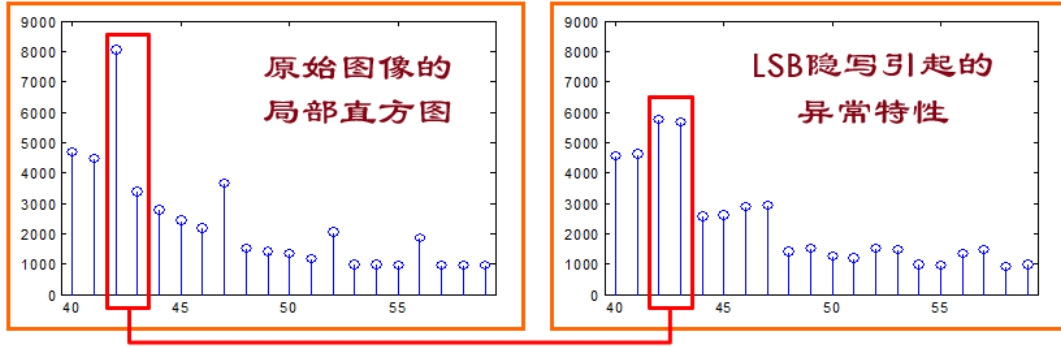


图 7 原始图像与隐写后像素值频次趋同现象图

卡方分析的代码如下所示：

```
def stgPrb(matrix):
    count = np.zeros(256, dtype=int)
    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            count[matrix[i, j]] += 1
    h2i = count[0::2]
    h2is = (h2i + count[1::2]) / 2
    filter = h2is != 0
    idx = np.where(filter)[0] # 获取非零值的索引
    r = np.sum(((h2i[idx] - h2is[idx]) ** 2) / h2is[idx])
    k = len(idx)
    p = 1 - chi2.cdf(r, k - 1)
    return r, p
```

利用 LSB 隐写所引发的统计的不对称性，导致相邻像素值对的频数分布发生改变，相较原始图像更加均匀。由直方图可以清晰的看出，在经过 LSB 隐写处理后，图像的值对会出现一种类似于削峰的状态，即相邻的值对的值会趋于一致的情况，由此可用于检测图像是否被隐写过。

以案例一图像为例，针对隐写率即 LSB 替换比例为 1、0.5、0 时，进行 LSB 隐写后进行卡方分析，观察 r 值与 P 值的变化。当隐写率为 0 时，即为原始图像，此时为未隐写状态，原始图像 r 值: 337.6170435673783, P 值: 0.0；当隐写率为 1 时，隐写图像 r 值: 60.99766143753139, P 值: 0.9999838751089152，此时 P 值接近于 1，此结果表明图像中高度疑似含有隐秘信息；当隐写率为 0.5 时，隐写图像的 r 值: 200.46750481853942, P 值: 7.726632430538061e-07，此时的 R 值与未隐写时相差较大，P 值几乎为 0 且灰度图无明显差异，因此不能判断出是否存在隐写。其隐写率与 r 值、P 值的关系及其结论如表 2 所示，在隐

写率为 1 时的案例一图像结果对比如图 8 所示。

表 2 隐写率、r 值、P 值数据结论表

隐写率	r 值	P 值	检测结论
0	337.6170435673783	0.0	未隐写
0.5	200.46750481853942	$7.726632430538061 \times 10^{-7}$	无法判断
1	60.99766143753139	0.9999838751089152	高度疑似隐写

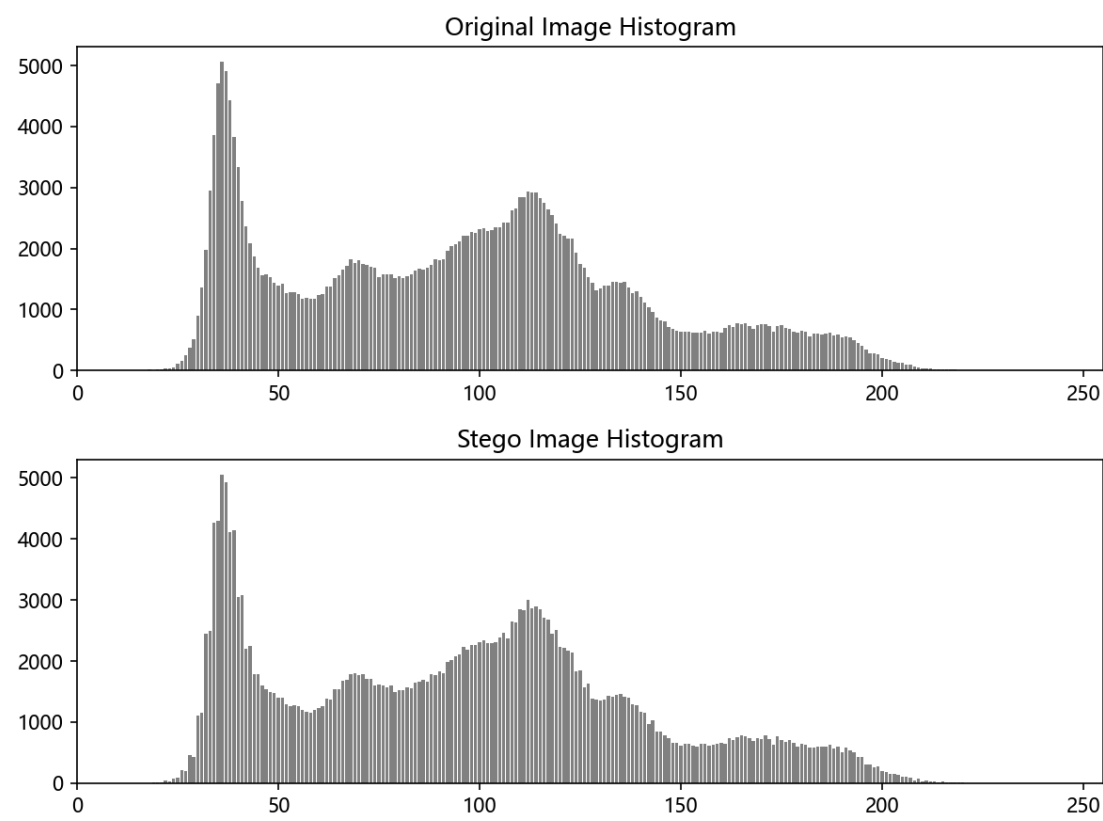


图 8 隐写率为 1 时的案例一图像像素值对图

对案例二的图像以隐写率为 1 时进行比对结果如下所示：隐写图像的峰值信噪比（PSNR）为：51.14 dB；原始图像的香农熵为：7.0097；隐写图像的香农熵为：7.0198；原始图像的 r 值: 501.5247491861817，P 值: 0.0；隐写图像的 r 值: 65.38506753996877，P 值: 0.9999954949915072。其隐写率与香农熵、r 值、P 值的关系及其结论如表 3 所示，在隐写率为 1 时的案例二图像结果对比如图 9 所示。

表 3 隐写率、r 值、P 值数据结论表

隐写率	r 值	P 值	香农熵	检测结论
0	501.5247491861817	0.0	7.0097	未隐写
1	65.38506753996877	0.9999954949915072	7.0198	高度疑似隐写

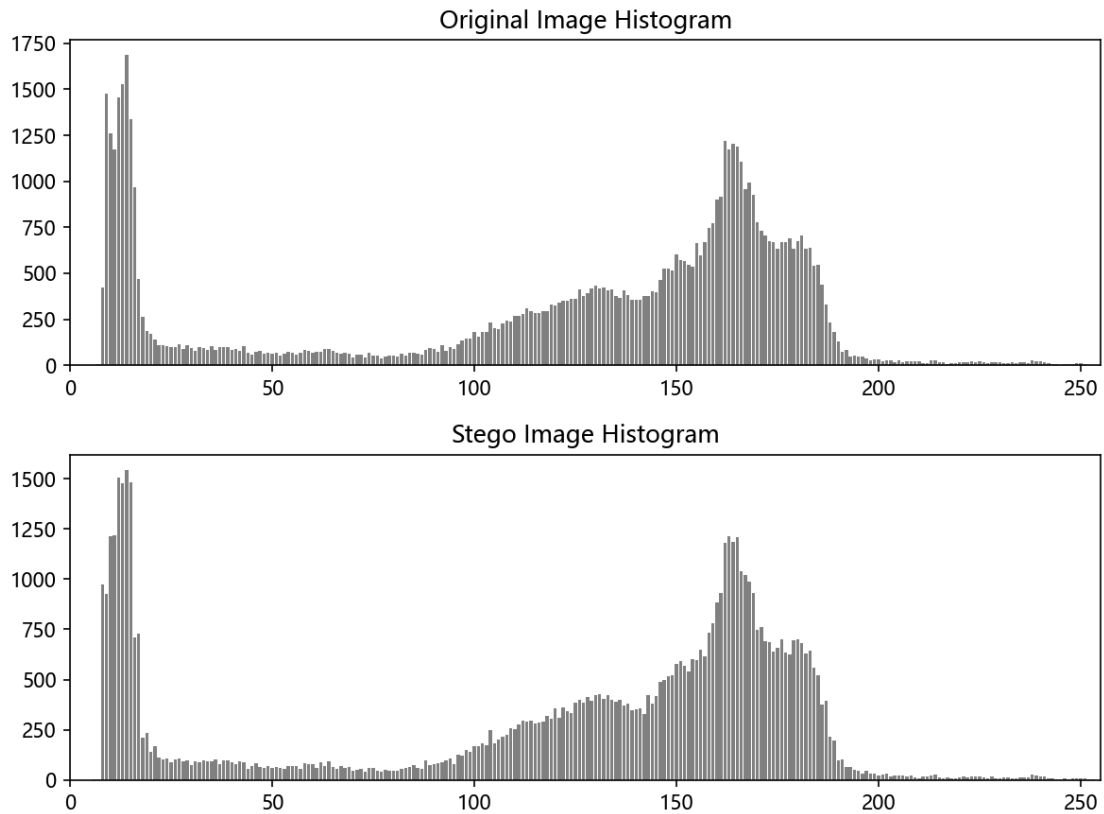


图 8 隐写率为 1 时的案例一图像像素值对图

由此可知，卡方检测在针对低隐写率场景，隐写仅修改少量像素时，此时 r 值下降不明显， P 值可能仍近似为 0，由此很可能会造成漏检的情况。由此，这种检测方式只有在几乎满嵌的情况下才能成功识别。此外，如果信息嵌入方式不是顺序嵌入，而是采取随机嵌入，卡方检验的方式同样难以生效。

3.2.2 信息量估计分析

信息量估计法是一种通过分析图像直方图的统计特征变化来检测隐写行为的方法，特别适用于 LSB 隐写。通过对直方图的分析，可以了解嵌入信息后图像的变化程度，从而估计隐写的质量与可检测性。其核心的假设在于，隐写将会破坏自然图像的像素值分布规律，从而导致直方图出现可量化的异常情况。

信息量估计法是通过计算直方图的偶数和奇数位置的像素值的差异来估计信息的嵌入，如果图像经过隐写处理，其直方图与原图的直方图可能会有所不同。通过计算 F_1 和 F_2 ，可以量化这些差异，并对比两次隐写前后直方图的变化判断是否曾被篡改。

在经过二次隐写的处理后，被处理图像将会有两个显著的特征。一方面，经过二次隐写后，整个图像的整体质量会减半；另一方面，所生成的两条曲线

(F₁,F₂)的细节里的质量会出现相互逼近，即两者之间的值呈现靠近的趋势。由此特征可说明图像经过重写后，其相邻的像素的质量会趋于相近，以此作为图像的隐写检测。与原图对比，信息量估计法分析如图 9 所示。

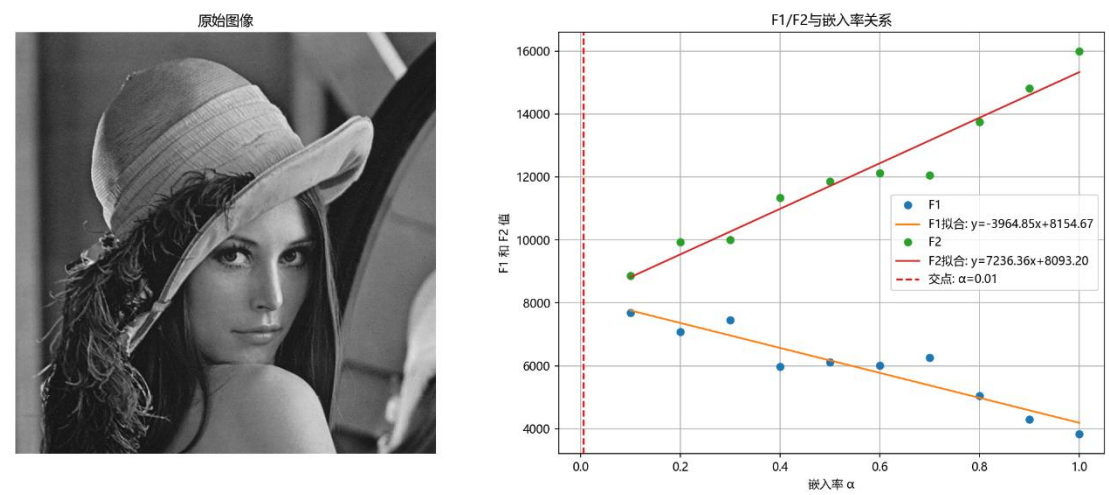


图 9.(a) 原图在不同嵌入值下一次隐写的 F₁、F₂ 变化图

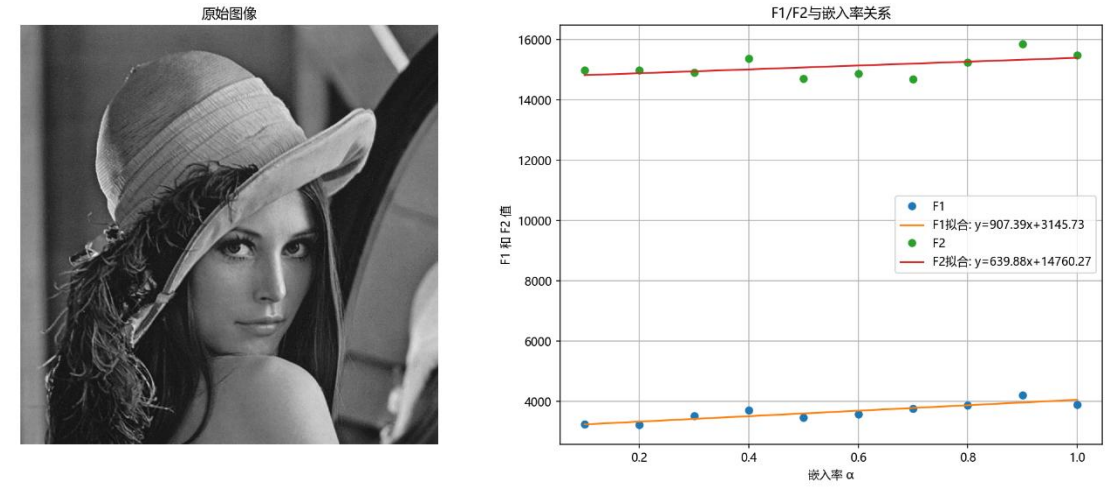


图 9.(b) 隐写后的图片在不同嵌入值下二次隐写的 F₁、F₂ 变化图

图 9 原图经二次隐写下不同嵌入值下的 F₁、F₂ 变化图

由图 9(a)纵坐标相差值可发现，在原图情况下即隐写率为 0 时的 F₁、F₂ 相互接近。随着隐写率的上升，F₁、F₂ 的差值逐渐增大，且 F₂ 明显大于 F₁。由图 9(b)可知二次隐写时 F₁、F₂ 的差值几乎保持恒定，相邻像素的质量趋于相近，满足前文所提到的二次隐写规律。F₁、F₂ 在原图的具体值和在第 1 次隐写时的具体值如表 4 所示，实验结果图附于表下。

表 4 隐写率、F₁ 值、F₂ 值数据结论表

隐写率	F ₁ 值	F ₂ 值	检测结论
0	7208	6872	未隐写
1	3540	15088	高度疑似隐写

信息量估计法的代码如下所示：

```
def CalF(matrix):
    # 将矩阵展平并计算直方图
    n, _ = np.histogram(matrix.ravel(), bins=256, range=(0, 256))
    n = n.astype(int) # 确保直方图的值是整数
    # 计算 h(2i), h(2i+1), h(2i+2)
    h2i = n[0:255:2] # h(2i)
    h2ip1 = n[1:256:2] # h(2i+1)
    h2ip2 = n[2:256:2] # h(2i+2)
    # 计算 f1 和 f2
    f1 = 0
    f2 = 0
    for i in range(127):
        f1 += abs(h2ip1[i] - h2i[i])
    for i in range(126):
        f2 += abs(h2ip2[i] - h2ip1[i])
    return f1, f2

def LSB(cover, rate):
    ste_cover = cover.astype(np.float64)
    m, n = ste_cover.shape
    # 根据 rate 计算秘密信息位的长度并生成秘密信息
    msg_len = int(np.floor(m * n * rate))
    msg = np.random.binomial(1, 0.5, msg_len) # 生成[0, 1]分布的随机数
    p = 0 # p 为秘密信息的位计数器
    # 按照产生的秘密信息依次将图片层的最后一位替换为秘密消息
    for f2 in range(n):
        for f1 in range(m):
            if p >= msg_len:
                break
            ste_cover[f1, f2] = (ste_cover[f1, f2] - ste_cover[f1, f2] % 2) + msg[p]
            p += 1
        if p == msg_len:
            break
    return ste_cover
```

3.2.3 RS 分析

隐写技术通过修改图像的某些像素或频率成分来嵌入数据，这些操作通常对图像的高频部分（如边缘信息、纹理细节等）影响较小，但很可能会影响到低频部分，例如隐写操作中 LSB（最低有效位）修改、DCT（离散余弦变换）嵌入等。隐写信息的嵌入通常会引入以下变化：

针对低频影响，隐写信息在频域的低频部分可能会引起数据的改变，尤其是隐写信息的嵌入位置选择和压缩比的不同；

针对高频影响，隐写操作可能会引入某些高频噪声，改变图像的原始频谱特征，从而可能导致图像边缘模糊；

针对频率的分布变化，正常图像在频域中具有特定的频率分布，而隐写信息的嵌入会打破这种原有的分布规律，这个现象同样是各类分析是否存在隐写方法的核心原理。

RS 分析是一种通过翻转像素最低位并统计块相关性变化判断隐写存在性。其核心在于可通过频域统计来检测隐写信息。通过 RS 分析法可以得到 rm 、 sm 、 r_m 和 s_m 随隐写率变化的数值，以原图为例分别展示这四条曲线的特征规律，其规律如图 10 所示。

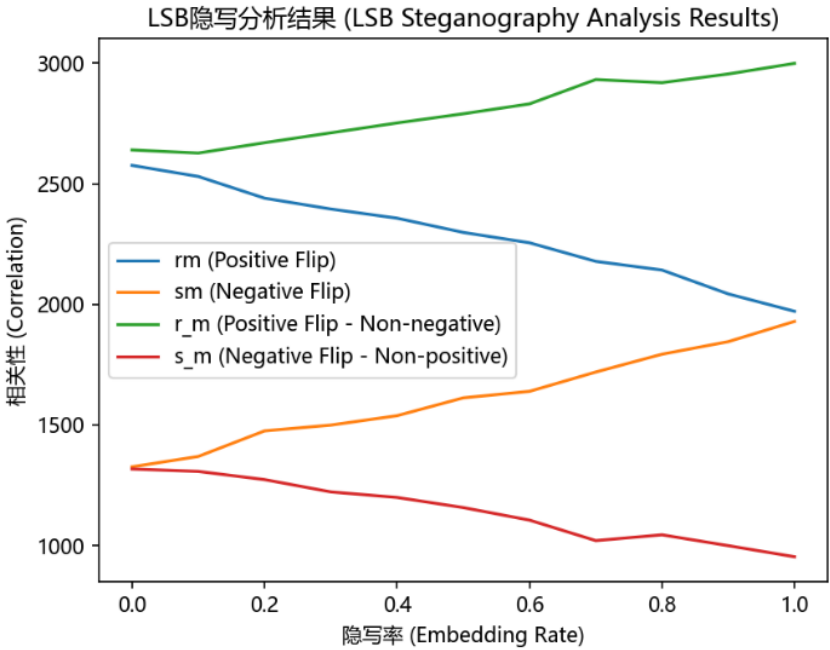


图 10 在原图情况下的 RS 分析所得四条曲线特征图

rm : 蓝色曲线，表示正向翻转的相关性得分。该曲线随着隐写率的增加，相关性得分呈下降趋势，表明隐写信息的增加导致图像规律性的减少；

sm : 橙色曲线，表示负向翻转的相关性得分。该曲线随着隐写率的增加而上升，表明负向翻转在高隐写率下更能保持图像的规律性；

r_m : 绿色曲线，表示正向翻转且非负向的相关性得分。该曲线随着隐写率的增加而上升，表明在高隐写率下，正向翻转的非负向特性变得更加明显；

s_m : 红色曲线，表示负向翻转且非正向的相关性得分。该曲线随着隐写

率的增加而下降，表明负向翻转的非正向特性在高隐写率下被削弱。

在隐写率为 0.9 时，针对二次隐写结果如图 11 所示。

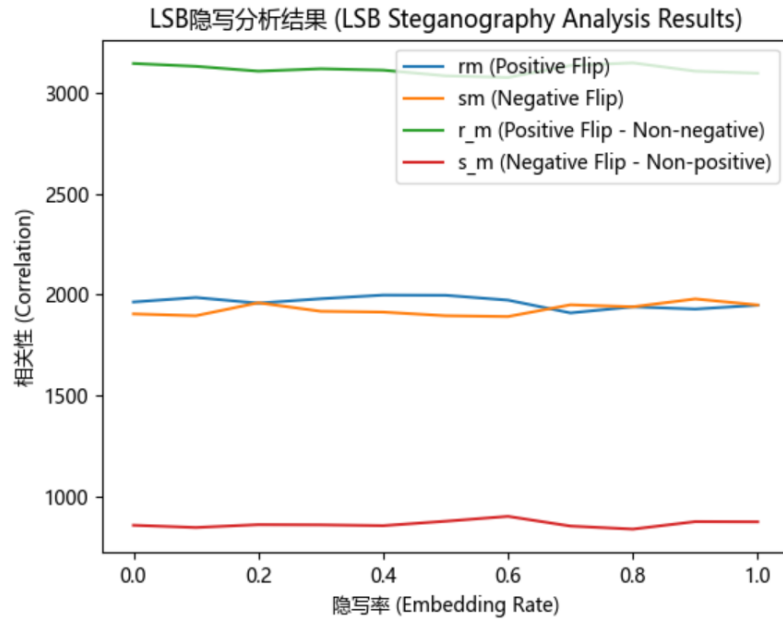


图 11 在二次隐写下的 RS 分析所得四条曲线特征图

由图 11 可知，正向翻转的相关性得分普遍下降，而负向翻转的相关性得分则上升，针对隐写后基本服从均匀分布。这意味着在 LSB 隐写过程中，隐写破坏原图像像素间的某些自然相关性，RS 分析可通过量化翻转操作的规律性变化来捕捉到这些原有像素被改写的变化。

RS 分析代码如下所示：（由于代码过多在下面只列出伪代码）

// 1. Z 字形扫描：将 8x8 矩阵按 Z 字形转换为 1D 数组

```
Function ZScan(matrix[8][8])
    output[64] = []
    i = 0, j = 0
    direction = "UP-RIGHT" // 初始方向
    For k = 0 To 63
        output[k] = matrix[i][j]
        If 当前在顶部边界 And 不在右上角
            j++ // 右移
            direction = "DOWN-LEFT"
        Else If 当前在右侧边界 And 不在右下角
            i++ // 下移
            direction = "DOWN-LEFT"
        Else If 当前在左侧边界 And 不在左下角
            i++ // 下移
            direction = "UP-RIGHT"
        Else If 当前在底部边界 And 不在右下角
```

```

        j++ // 右移
        direction = "UP-RIGHT"
    Else If direction == "UP-RIGHT"
        i--, j++ // 右上移动
    Else // direction == "DOWN-LEFT"
        i++, j-- // 左下移动
    End If
End For
Return output
End Function

```

// 2. 计算像素相关性（相邻像素绝对差之和）

```

Function CalculateCorrelation(array[64])
    sum = 0
    For i = 0 To 62
        sum += |array[i+1] - array[i]|
    End For
    Return sum
End Function

```

// 3. 翻转函数

```

Function F1(val) // 正翻转：偶数+1，奇数-1
    If val 是偶数 Then Return val + 1 Else Return val - 1
End Function

```

```

Function F_1(val) // 负翻转：偶数-1，奇数+1
    If val 是偶数 Then Return val - 1 Else Return val + 1
End Function

```

// 4. 生成随机模式

```

Function GenerateRandomPattern(type)
    pattern[64] = []
    For i = 0 To 63
        If type == 1 Then
            pattern[i] = Random(0, 1) // 生成 0 或 1
        Else
            pattern[i] = Random(-1, 0) // 生成 -1 或 0
        End If
    End For
    Return pattern
End Function

```

// 5. 单块 RS 分析

```

Function AnalyzeBlock(block[8][8])

```

```

// 计算原始相关性
originalArray = ZScan(block)
originalCorr = CalculateCorrelation(originalArray)
// 应用 F1 翻转
f1Pattern = GenerateRandomPattern(1)
f1Block = ApplyPattern(block, f1Pattern, F1)
f1Array = ZScan(f1Block)
f1Corr = CalculateCorrelation(f1Array)
// 应用 F_1 翻转
f_1Pattern = GenerateRandomPattern(-1)
f_1Block = ApplyPattern(block, f_1Pattern, F_1)
f_1Array = ZScan(f_1Block)
f_1Corr = CalculateCorrelation(f_1Array)
rm = If(originalCorr < f1Corr) Then 1 Else 0
sm = If(originalCorr > f1Corr) Then 1 Else 0
r_m = If(originalCorr < f_1Corr) Then 1 Else 0
s_m = If(originalCorr > f_1Corr) Then 1 Else 0
Return [rm, sm, r_m, s_m]
End Function

// 6. 主函数: RS 隐写率估计
Function RSEstimation(imagePath, iterations = 20)
    image = ReadImage(imagePath)
    rows = image.height / 8
    cols = image.width / 8
    totalEstimatedRate = 0.0
    For iter = 1 To iterations
        rmSum = smSum = r_mSum = s_mSum = 0
        // 遍历所有 8x8 块
        For i = 0 To rows-1
            For j = 0 To cols-1
                block = Extract8x8Block(image, i*8, j*8)
                [rm, sm, r_m, s_m] = AnalyzeBlock(block)
                rmSum += rm, smSum += sm, r_mSum += r_m, s_mSum += s_m
            End For
        End For
        // 计算差异值
        d0 = rmSum - smSum
        d1 = rmSum - smSum
        d2 = r_mSum - s_mSum
        d3 = r_mSum - s_mSum
        // 解二次方程:  $2(d1+d0)x^2 + (d2-d3-d1-3d0)x + (d0-d2) = 0$ 
        a = 2 * (d1 + d0)
        b = d2 - d3 - d1 - 3 * d0
    End For
End Function

```

```

c = d0 - d2
roots = SolveQuadratic(a, b, c)
estimatedRate = If(roots[0] > roots[1])
                    Then roots[1] / (roots[1] - 0.5)
                    Else roots[0] / (roots[0] - 0.5)
totalEstimatedRate += estimatedRate
End For
Return totalEstimatedRate / iterations
End Function

```

4、总结与收获

LSB 隐写算法通过将载体图像像素值与秘密信息均转换为二进制形式，并逐位替换像素最低有效位（LSB）实现信息嵌入。由于其视觉隐蔽性较强，人眼难以察觉图像异常，同时也具备实现简单、隐藏容量大的优势。

然而，该算法鲁棒性较差，易受压缩、裁剪等操作破坏隐藏信息，且在嵌入大量数据时，视觉保真度仍处于较高程度，而具有良好的视觉隐蔽性。但可能因统计特征暴露而被卡方检测、RS 分析等工具有效识别载体中是否存在隐秘信息。

在信息隐藏的核心指标中，LSB 隐写体现了不可感知性、鲁棒性与数据容量存在一定关系。不可供执行指隐写后载体无明显变化，鲁棒性指抵抗干扰的能力，数据容量指可嵌入的信息量。通常而言，高容量与强隐蔽性通常以牺牲鲁棒性为代价，实际应用中需根据场景需求进行权衡，以达到最终更好的隐写效果。